

PIS-IN AOI AI 智能质检技术方案

V3.5 单机实施与生产门禁版

项目周期 2024.09 - 2025.01

版本 V3.5 | 状态 可运行 PoC / 生产门禁明确

一、交付结论与边界

V3.5 已形成可运行的单机 PoC：操作员或本机 AOI 软件触发采集，FastAPI + React + PostgreSQL 完成数据、推理、复核和报表闭环；Agent/RAG 作为独立辅助服务接入。自动化 Handler 暂不接入，相关接口默认关闭。

63	6	3	1
API 测试	Agent/RAG 测试	前端单测	E2E 主流程

已实现

- 24 条确定性演示事件、PASS/FAIL/REVIEW 三态决策、Tray Map、复核写回、工站预警、异常报告和模型治理页面。
- Source Key 采用 DeviceID + DeviceSessionID + InspectionSequence + TrayID + SlotIndex + Surface, Light ID 仅作为附件属性。
- 真实 PIS-IN 导入接口、TensorRT 可替换适配器、Docker Compose 与 GPU overlay 已提供；Handler 通过 HANDLER_INTEGRATION_ENABLED=false 默认关闭。

必须如实披露

- 当前仓库没有真实 AOI 图片、标注集和 YOLO 权重；YOLOv8/TensorRT 的真实精度与端到端吞吐需现场数据复测。
- Compose 单机服务可启动并通过健康检查；这证明软件部署链路可复现，不代表相机、TensorRT Engine 和连续生产验收已完成。
- 质量指标采用分阶段目标：原 AOI NG 候选池误报基线 12%，PoC <=6%，单机受控上线 <=3%，成熟阶段 <=1.5%；全检口径单独以 <=0.5% 管理。

二、项目背景与业务目标

PIS-IN AOI 在本机产生 2D 多光源图像、3D 量测和原 AOI 结果。当前由操作员或本机软件触发，不依赖 Handler。项目增加可追溯的 AI 旁路复判层，解决多文件乱序、重复、缺失、设备重启计数归零和异常处置无法闭环的问题。



主要业务流程

- 采集文件稳定性检查 -> 解析身份 -> 生成确定性 Source Key -> 以 event_uuid 聚合多光源附件。
- 附件完整且校验通过后进入推理；未知、身份缺失、错配、低置信度和缺失关键输入统一进入 REVIEW 或 QUARANTINED。
- 工站缺陷率达到滑动窗口门槛后生成告警；人工确认告警后生成 DRAFT 异常报告，质量/工艺人员完成最终处置。

三、总体技术架构与数据流



架构原则

- 当前采用单机容器化模块服务架构：原 AOI 继续承担基础检测，AI 在同一工控机完成旁路复判，Handler 自动握手与实时 BIN 回传默认关闭。
- 业务 API、持久化、推理适配器和 Agent/RAG 均以可替换接口隔离；Agent 超时只降级为确定性草稿，不阻断检测看板。
- 数据从事件、附件、模型结果、复核、告警到异常报告全程留有 event_uuid、模型版本、策略版本和证据引用；MES 仅可选异步同步。

数据流向

PIS-IN 文件 / AOI 模拟器 -> PIS-IN Source Adapter -> Source Key / 附件校验 -> InspectionEvent -> DemoInferenceAdapter 或 TensorRtInferenceAdapter -> 三态决策 -> PostgreSQL -> React 看板 / 工站告警 / Agent 离线治理。

四、数据关联一致性与状态机

Source Key 规范化

规范化 JSON 按 key 排序并使用紧凑分隔符后 SHA-256，避免下划线拼接冲突。设备重启通过 DeviceSessionID 隔离重复计数；R/G/B/RING 不拆分事件，均关联到同一 event_uuid。

```
{"device_id":"PIS-01","device_session_id":"BOOT-77","inspection_sequence":"1042","tray_id":"TRAY-09","slot_index":"A07","surface":"TOP"}
```

状态	进入条件	可转换 / 处理
RECEIVED / COLLECTING	首个文件到达，身份已解析	等待附件；超时进入 EXPIRED
READY / VALIDATED	必需光源、3D/AOI 和文件校验通过	进入推理；校验失败 REVIEW_REQUIRED
INFERRED	推理与规则决策完成	ARCHIVED 或 REVIEW_REQUIRED
QUARANTINED	身份缺失或无法归属	人工确认、重处理或丢弃
ARCHIVED / INVALID / EXPIRED	结果归档或终态	不可自动回到实时链路

幂等约束

inspection_events.source_key_hash 唯一；附件以 event_uuid + data_type + light_id + file_hash 去重；推理结果以 event_uuid + model_version + policy_version + input_fingerprint 共存，支持新旧模型影子对比。

05 / DATA MODEL

五、核心数据模型与 API

实体	职责	关键字段 / 约束
inspection_events	一次物理检测事件	event_uuid、source_key_hash、设备/Tray/Slot、三态决策
data_attachment_links	多光源、3D、AOI 附件	light_id、file_hash、quarantine_id、唯一去重
inference_results	模型版本化结果	model_version、policy_version、bbox、3D 摘要、延迟
quarantine_events	身份缺失隔离	quarantine_id、解析错误、部分字段、处理结果
review_records	人在回路金标准	人工决策、缺陷码、备注、复核人
station_alerts / anomaly_reports	工站推送与异常闭环	窗口缺陷率、阈值、确认人、报告证据

已验证接口

- GET /api/v1/dashboard/summary、/inspections、/trays/{tray_id}、/reviews、/alerts、/reports、/model-releases。
 - POST /api/v1/inspections/import/pis-in： 正常模式规范化附件并幂等写入； 身份缺失返回 202 + quarantine_id。
 - POST /api/v1/reviews、/alerts/{id}/acknowledge、/reports： 操作持久化， 页面刷新后仍可追溯。
 - Agent
- 内部端点： /agent-api/v1/health、knowledge/search、assess-data-quality、draft-report、recommend-model-release。

六、YOLOv8 与 TensorRT 实施方案

模型选型

PoC 采用 YOLOv8s 作为速度与精度的平衡基线；按缺陷尺寸、漏检风险和边缘 GPU 资源再决定升级 YOLOv8m/l/x。模型输出不直接决定 PASS，必须经过输入完整性、3D 硬规则和置信度策略层。

阶段	模型 / 运行时	验收参数
GPU-free 演示	DemoInferenceAdapter	确定性输出、21-28ms 模拟延迟，仅验证业务闭环
PoC 联调	YOLOv8s ONNX / TensorRT FP16	mAP50、召回率、关键缺陷漏放率、P95、吞吐
生产候选	YOLOv8s/m + TensorRT Engine	盲测、影子差异率、回滚演练、静默错配率=0
升级条件	YOLOv8m/l/x	仅当小缺陷召回、遮挡场景或多尺度误检经数据证明需要

推理链路

预处理 -> 多光源通道组织 -> YOLO 检测框/置信度 -> 3D 量测摘要 -> 规则护栏 -> PASS / FAIL / REVIEW。TensorRtInferenceAdapter 在 engine 缺失或 runtime 不可用时 fail-closed，返回不可用状态，不伪造 PASS。

必须采集的真实参数

- 按工站/产品/缺陷类别统计 precision、recall、F1、关键缺陷漏放率和人工复核率。
- 记录模型版本、输入 fingerprint、P50/P95 延迟、持续吞吐、队列积压、GPU 显存和温度。
- 以全检件为分母的误报率与 AOI NG 候选池误报率分开统计，不能用一个 6% 覆盖所有风险。

七、PASS / FAIL / REVIEW 决策与人在回路



规则优先级	条件	结果
1	身份缺失、错配、关键输入缺失	REVIEW，或 QUARANTINED
2	3D 硬限超限或缺陷分数达到门槛	FAIL + defect_code
3	完整输入且正常置信度达到策略阈值	PASS
4	其余低置信度、未知缺陷、Agent 不可用	REVIEW

人工复核 vs 人在回路

人工复核是具体操作：对 REVIEW 事件查看图像、框、3D、原 AOI 结果并提交 PASS/FAIL、缺陷码和备注；人在回路是治理边界：人拥有最终确认权，金标准回流、模型发布、异常报告结论和高风险放行均不能由 Agent 自动替代。

八、3 个 Agent 与 1 套 RAG

Agent	输入 / 输出	边界
数据质量 Agent	附件、Source Key、时间偏差、重复与缺失 -> 风险等级、事实、处置建议	不能修改事件身份，不能创建自动 PASS
复核与异常报告 Agent	告警窗口、代表事件、缺陷码、历史案例 -> DRAFT 报告与开放问题	事实与假设分开；必须人工确认
模型治理 Agent	召回、漏放、误报、P95、吞吐、盲测、回滚、静默错配 -> 发布建议	只能 BLOCK / SHADOW_MORE / READY_FOR_APPROVAL，不能执行发布

RAG 知识库

PostgreSQL + pgvector 保存缺陷字典、SOP、设备手册、历史异常报告和模型发布说明。检索结果携带 document_id、chunk_id、category 和 citation；确定性 Demo embedding 保障无 LLM 时可演示，生产可替换为企业 embedding 服务。

故障隔离

主 API 通过超时客户端调用 Agent。Agent 超时、连接失败或引用校验失败时，报告仍落库为 DRAFT 并标记 agent_status=UNAVAILABLE；实时检测、看板和复核 API 不被拖垮。

九、缺陷报表、工站推送与异常报告

可行性结论

高缺陷率工站推送与异常报告是可执行的质量运营闭环，但必须采用最小样本数 + 滑动窗口 + 人工确认三道门，避免单件异常造成误报警。当前服务已实现窗口缺陷率计算、OPEN/ACKNOWLEDGED/CLOSED 状态和 DRAFT 报告持久化。

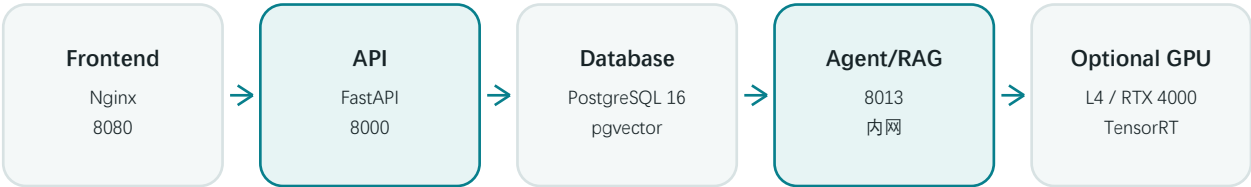
报表维度	示例指标	用途
缺陷分类	BALL_BRIDGE、SOLDER_VOID、COPLANARITY、MISSING_BALL、UNKNOWN	定位材料、设备、工艺和模型问题
工站推送	窗口样本数、FAIL 率、阈值、连续异常窗口	推送班组长、质量、工艺
异常报告	事实、代表事件、证据引用、开放问题、处理状态	形成 8D / CAPA 输入
模型反馈	人工金标准、误报、漏放、影子差异	数据闭环与版本治理

建议生产门槛

- 窗口样本数 ≥ 20 （按产线节拍调整），缺陷率 $\geq 8\%$ 才进入 OPEN；连续异常窗口可提高等级。
- 告警确认后才允许生成报告草稿；报告批准、关闭和恢复均需角色与时间戳。
- 报告内事实必须来自事件与检索证据，推测必须写为开放问题，避免自动归因。

10 / DEPLOYMENT

十、部署、算力与环境切换



模式	配置	用途
GPU-free demo	PostgreSQL + API + simulator + frontend + deterministic Agent/RAG	本地演示、培训、接口联调
local-llm	Agent/RAG overlay 请求 1 张 L4 类 GPU，OpenAI-compatible endpoint	离线报告和知识问答质量验证
tensorrt	边缘 GPU 运行 TensorRT Engine，保留同一 API 与决策护栏	影子运行和受控上线
推荐 PoC	训练 2 x L40S 48GB；边缘 2 台 RTX 4000 Ada；Agent/RAG 1 x L4；40-80TB NAS	标准估算，须现场核算

启动与回退

Compose 基础模式在一台工控机运行且不请求 GPU；GPU overlay 仅在 local-llm/tensorrt profile 请求 GPU。TensorRT engine 缺失时自动回到 REVIEW 或停用旁路，不允许静默放行。Handler 路由默认不注册；数据库迁移使用 Alembic，服务通过健康检查和 restart policy 管理。

十一、安全、可靠性与审计

风险	控制措施	验证证据
乱序 / 重复文件	Source Key + hash 幂等，附件汇聚后再推理	导入重复测试、状态机测试
设备重启计数归零	DeviceSessionID / boot id 隔离	跨 session 同序号测试
身份缺失 / 错配	quarantine_events 持久化，人工归属	202 状态与 quarantine_id
Agent 超时	连接/总超时、确定性草稿降级、实时链路隔离	agent_failure_isolation 测试
模型误放	REVIEW 默认、发布门禁、回滚演练、静默错配=0	治理 Agent BLOCK 测试
数据泄露	Agent 内网暴露，不经 Nginx 发布；日志脱敏与角色鉴权待现场接入	部署配置审计

审计最小字段

event_uuid、source_key_hash、输入
fingerprint、model_version、policy_version、decision、reason_code、reviewer、acknowledged_by、report_id、Agent
引用和时间戳。

十二、测试与实测结果

层级	结果	覆盖
FastAPI / 领域	63 passed	Source Key、三态决策、导入幂等、隔离、告警、Handler 默认关闭
Agent/RAG	6 passed	检索引用、数据质量、报告草稿、发布门禁、API
模拟器	2 passed	确定性场景与 payload
React	3 passed + typecheck + build	导航、Tray Map、复核交互、ECharts 按需构建
Playwright	1 passed	复核 -> Tray 追溯 -> 告警 -> DRAFT 报告
Compose	运行健康 PASS	单机服务、端口、健康检查、无 GPU 基础模式

当前可量化结果

24 确定性演示事件	16/4/4 PASS / FAIL / REVIEW	8 前端运营页面	0 自动发布动作
----------------------------------	---	--------------------------------	--------------------------------

尚未宣称的结果

真实 YOLO 精度、现场端到端 P95、持续吞吐、生产误报率、漏放率和静默错配率必须用连续生产样本和独立盲测完成；当前测试证明的是软件链路和安全护栏，不是现场验收结论。

十三、生产上线门禁与路线图

阶段	进入条件	退出条件
设计评审	V3.5 架构、Source Key、DDL 和状态机确认	接口字段清单签字
PoC 数据审计	连续生产样例、身份字段、光源集合可获得	错配 / 重复 / 延迟 / 断网注入通过
YOLO/TensorRT 联调	真实标注集、engine、GPU 环境	盲测召回、关键缺陷漏放、P95 和吞吐达标
影子运行	新旧模型共存、报告与告警可回溯	静默错配率=0，差异可解释，回滚演练通过
生产自动 PASS	质量、设备、工艺共同批准	持续监控、应急停用、审计与培训齐备

后续优化

- 接入真实 PIS-IN manifest / 文件监听与鉴权，补齐多工站、多产品和夜班异常统计。
- 用人工复核金标准做 hard-negative mining 和阈值校准；按缺陷类别独立设定风险门槛。
- 完善 Prometheus 指标、日志检索、告警通知和模型回滚操作审计。
- 对 Agent/RAG 增加知识版本、引用命中率、回答拒答率和人工采纳率指标。

十四、面试与项目答辩口径

一句话介绍

我负责把 PIS-IN AOI 的多源数据关联、AI 旁路复判和质量运营闭环做成可在一台工控机运行的 V3.5 PoC：本机触发采集，用 Source Key 解决事件一致性，用 YOLOv8/TensorRT 承载推理，用三态决策与人在回路控制风险，再用 3 个 Agent + pgvector RAG 生成可引用的治理建议。

追问	回答重点
为什么不是 Agent 直接判定？	实时链路必须确定性、低延迟、可回退；Agent 只做解释、检索和报告草稿，最终放行由规则和人确认。
人工复核和人在回路区别？	前者是 REVIEW 队列中的具体动作；后者是整个系统的责任边界和审批权。
6% 是否代表生产效果？	不是。6% 是 PoC 目标，生产需用现场分母、盲测和漏放/静默错配共同验收。
为什么先用 YOLOv8s？	以边缘资源和实时性为约束的基线；是否升级 m/l/x 由小缺陷召回和 P95 数据决定。
如何证明可落地？	真实 API、持久化、故障隔离、导入幂等、E2E 主流程和 Compose 部署合同均可复现；现场门禁仍保留。

十五、附录：项目固定口径

项目	固定口径
周期	2024.09 - 2025.01
团队	8 人按阶段参与：产品/AI 负责人 1、后端 2、前端 1、算法 2、测试 1、实施运维 1
运行	AOI 工控机单机独立运行；Handler 默认关闭；MES 仅可选异步同步
Agent	数据质量 Agent、复核与异常报告 Agent、模型治理 Agent，共 3 个；不进入实时自动 PASS
RAG	PostgreSQL + pgvector；缺陷字典、SOP、设备手册、历史异常报告、模型发布说明
质量目标	AOI NG 候选池误报：基线 12% -> PoC <=6% -> 单机受控 <=3% -> 成熟 <=1.5%；全检口径 <=0.5% 单列
版本状态	V3.5：可运行单机 PoC / 数据适配与部署基线；自动 PASS 待现场门禁

文档结束